

Lisp Recursion

CS-3180

Functional Programming

- List is built for functional programming
- Loops are inherently procedural
- Lisp has "loop" style functions
- That we will not cover
- Everything is done via recursion

Recursion

- Do some simple recursive problem
- Recursion takes a few forms:
- Recursive Definitions (e.g. math problems)
- Recursing over linear structures (e.g. `car`, `cdr`)
- Recursing over self-similar structures (e.g. trees)

Recursion

- Do some simple recursive problem
- Recursion takes a few forms:
- Recursive Definitions (e.g. math problems)
- Recursing over linear structures (e.g. `car`, `cdr`)
- Recursing over self-similar structures (e.g. trees)
- We won't do a lot of this last one

Factorial

- You have likely seen factorial done before
- Good practice to see
- Get into the recursive mindset

```
1 (defun factorial (n)
2   (if (<= n 1)
3       1
4       (* n (factorial (- n 1)))))
5
6 (format t "N! was ~S~%" (factorial 3))
```

Get Penultimate

- Missing some edge cases
- No explicit returns
- Can make guard clauses harder

```
1 (defun get-penultimate (s)
2   (let ((cur (car s)))
3     (if (null (cdr (cdr s)))
4         cur
5         (get-penultimate (cdr s)))))
6
7
8 (format t "penultimate was ~S~%" (get-penultimate '(ALICE BOB
```

Defun Namespace

- Consider the namespace here
- `factorial` can call `factorial`
- `factorial` is defined both within AND outside the function
- Keep this scope in mind as we move forward

Flet

- Allows you to scope functions
- First parameters is ALL your functions you want to scope
- Body is the scope that allows you to call those functions
- CANNOT be recursive
- CANNOT call each other

```
(flet (functions...) (code that calls those functions))
```

Flet

- Entire form evaluates to the body
- Which calls your local functions

```
1 (format t "~S~%"
2   (flet ((double (value)
3         (* value 2))
4         (square (value)
5               (* value value))
6         (triple (value)
7               (* value value value))))
8   (triple (double (square 3))))
```

Labels

- Same as `flet`
- Except the functions can be recursive
- They can also call each other
- Often just used for simple recursive functions

```
1 (labels ((factorial (n)
2           (if (<= n 1)
3               1
4               (* n (factorial (- n 1))))))
5   (factorial 5))
```

Labels

- Canonical example for labels
- You'll likely need this on your project

```
1 (defun parity (value)
2   (labels ((is-even (x)
3             (if (zerop x)
4                 t
5                 (is-odd (- x 1))))
6             (is-odd (x)
7                     (if (zerop x)
8                         nil
9                         (is-even (- x 1)))))
10  (if (is-even value)
11      'even
12      'odd)))
13
14 (format t "~S~%" (parity 9))
```

Tail Calls

- Recursion is expensive
- Every new call -> new stack frame
- Load all the data each time
- In most functional languages, some additional optimizations occur
- Under specific conditions
- Let's start in a non-FP language

Tail Calls

- Javascript Example
- The last thing this function does is wait for `factorial` to finish
- EVERY stack frame must resolve before we return

```
1 function factorial(n) {  
2   if (n === 1) return 1;  
3   return n * factorial(n - 1);  
4 }
```

Tail Calls

- If the LAST thing the function did
- Was recursive call
- We wouldn't need to return address of the previous function
- We wouldn't have to wait
- We would need a way to pass all the work forward
- Commonly done via a parameter

Tail Calls

- VERY last thing we do is a recursive call
- Executor can re-use the same stack frame
- New parameter becomes previous return value
- Make recursion as fast as iteration

```
1 function factorial(n, accumulator = 1) {  
2   if (n === 0) return accumulator;  
3   return factorial(n - 1, n * accumulator); // tail call  
4 }
```

Tail Calls in Lisp

- This idea is critical for writing fast Lisp code
- Prefer tail calls if possible
- Harder to visualize
- (`&optional` is special form that allows optional + default params)
- Porting the code, you get something like this:

```
1 (defun factorial (n &optional (acc 1))  
2   (if (= n 0)  
3       acc  
4       (factorial (- n 1) (* n acc))))
```

Summary

- Functions are a little more complicated than just `defun`
- Scope function(s) with `flet`
- Use `labels` if you need them to see/call each other
- Tail Call Optimization allows executors to reuse the last stack frame

Questions?